

bqa0vtozh

February 17, 2026

```
[ ]: import numpy as np
import pandas as pd
```

```
[2]: data_csv=pd.read_csv("/kaggle/input/tree-nuts-image-classification/tree_nuts.
↳CSV")
```

```
[ ]: data_csv.head(5)
```

```
[ ]:      class index      filepaths  labels data set
0         0  train/almonds/001.jpg  almonds   train
1         0  train/almonds/002.jpg  almonds   train
2         0  train/almonds/003.jpg  almonds   train
3         0  train/almonds/004.jpg  almonds   train
4         0  train/almonds/005.jpg  almonds   train
```

```
[4]: print(f"Total number of Classes is : {len(data_csv.labels.unique())}")
```

Total number of Classes is : 10

```
[ ]: import matplotlib.pyplot as plt
from os import listdir
from sklearn.preprocessing import LabelBinarizer
import random
from matplotlib.image import imread
```

```
[6]: plt.figure(figsize=(20,20))
path = "/kaggle/input/tree-nuts-image-classification/train/"
for i in range(1,26):
    plt.subplot(5,5,i)
    plt.tight_layout()
    selected_folder=random.choice(sorted(listdir(path)))
    rand_img = imread(path +selected_folder+ '/' +random.
↳choice(listdir(path+selected_folder)))
    plt.imshow(rand_img)
    plt.title(str(selected_folder))
    plt.xlabel(rand_img.shape[1], fontsize = 10)
    plt.ylabel(rand_img.shape[0], fontsize = 10)
```



```
[7]: plt.figure(figsize=(20,20))
path = "/kaggle/input/tree-nuts-image-classification/test/"
for i in range(1,26):
    plt.subplot(5,5,i)
    plt.tight_layout()
    selected_folder=random.choice(sorted(listdir(path)))
    rand_img = imread(path +selected_folder+ '/' +random.
↳choice(listdir(path+selected_folder)))
    plt.imshow(rand_img)
    plt.title(str(selected_folder))
    plt.xlabel(rand_img.shape[1], fontsize = 10)
    plt.ylabel(rand_img.shape[0], fontsize = 10)
```



```
[8]: plt.figure(figsize=(20,20))
path = "/kaggle/input/tree-nuts-image-classification/valid/"
for i in range(1,26):
    plt.subplot(5,5,i)
    plt.tight_layout()
    selected_folder=random.choice(sorted(listdir(path)))
    rand_img = imread(path +selected_folder+ '/' +random.
↳choice(listdir(path+selected_folder)))
    plt.imshow(rand_img)
    plt.title(str(selected_folder))
    plt.xlabel(rand_img.shape[1], fontsize = 10)
    plt.ylabel(rand_img.shape[0], fontsize = 10)
```



```
[9]: from PIL import Image
      from keras.preprocessing.image import img_to_array, array_to_img
      from keras.preprocessing import image
```

```
[ ]: training_image_list, training_label_list=[], []
```

```
[ ]: main_directory="/kaggle/input/tree-nuts-image-classification/train/"
      list_main_dir=listdir(main_directory)
      for each in list_main_dir:
          imageslist=listdir(main_directory+'/'+each+'/')
          print(f"Images of {each } directory is loading...")
          for each_images in imageslist:
```

```

i=Image.open(f'{main_directory}/{each}/{each_images}')
i=i.resize((224 ,224 ))
image_array_format=img_to_array(i)
training_image_list.append(image_array_format)
training_label_list.append(each)

```

Images of hazelnuts directory is loading...
 Images of pistachios directory is loading...
 Images of pecans directory is loading...
 Images of coconut directory is loading...
 Images of macadamia directory is loading...
 Images of brazil nuts directory is loading...
 Images of cashews directory is loading...
 Images of almonds directory is loading...
 Images of pine nuts directory is loading...
 Images of walnuts directory is loading...

```

[12]: total_Image_category_count=pd.DataFrame(training_label_list).value_counts()
total_Image_category_count

```

```

[12]: pecans          161
brazil nuts        138
cashews            136
hazelnuts          120
pistachios         110
almonds            109
macadamia          101
pine nuts           98
walnuts            96
coconut            94
dtype: int64

```

```

[ ]: lb = LabelBinarizer()
train_y = lb.fit_transform(training_label_list)
print(lb.classes_)
train_y[1]

```

```

['almonds' 'brazil nuts' 'cashews' 'coconut' 'hazelnuts' 'macadamia'
 'pecans' 'pine nuts' 'pistachios' 'walnuts']

```

```

[ ]: array([0, 0, 0, 0, 1, 0, 0, 0, 0, 0])

```

```

[ ]: Validation_image_list,Validation_label_list=[],[]

```

```

[ ]: main_directory="/kaggle/input/tree-nuts-image-classification/valid/"
list_main_dir=listdir(main_directory)
for each in list_main_dir:

```

```

imageslist=listdir(main_directory+'/'+each+'/')
print(f"Images of {each} directory is loading...")
for each_images in imageslist:
    i=Image.open(f'{main_directory}/{each}/{each_images}')
    i = i.resize((224 ,224 ))
    image_array_format=img_to_array(i)
    Validation_image_list.append(image_array_format)
    Validation_label_list.append(each)

```

```

Images of hazelnuts directory is loading...
Images of pistachios directory is loading...
Images of pecans directory is loading...
Images of coconut directory is loading...
Images of macadamia directory is loading...
Images of brazil nuts directory is loading...
Images of cashews directory is loading...
Images of almonds directory is loading...
Images of pine nuts directory is loading...
Images of walnuts directory is loading...

```

```

[16]: total_Image_category_count=pd.DataFrame(Validation_label_list).value_counts()
total_Image_category_count

```

```

[16]: almonds      5
brazil nuts      5
cashews          5
coconut          5
hazelnuts        5
macadamia        5
pecans           5
pine nuts        5
pistachios       5
walnuts          5
dtype: int64

```

```

[17]: lb = LabelBinarizer()
validation_y = lb.fit_transform(Validation_label_list)
print(lb.classes_)
validation_y[1]

```

```

['almonds' 'brazil nuts' 'cashews' 'coconut' 'hazelnuts' 'macadamia'
 'pecans' 'pine nuts' 'pistachios' 'walnuts']

```

```

[17]: array([0, 0, 0, 0, 1, 0, 0, 0, 0, 0])

```

```

[18]: Test_image_list,Test_label_list=[],[]

```

```
[ ]: main_directory="/kaggle/input/tree-nuts-image-classification/test/"
list_main_dir=listdir(main_directory)
for each in list_main_dir:
    imageslist=listdir(main_directory+'/'+each+'/')
    print(f"Images of {each } directory is loading...")
    for each_images in imageslist:
        i=Image.open(f'{main_directory}/{each}/{each_images}')
        i = i.resize((224 ,224 ))
        image_array_format=img_to_array(i)
        Test_image_list.append(image_array_format)
        Test_label_list.append(each)
```

```
Images of hazelnuts directory is loading...
Images of pistachios directory is loading...
Images of pecans directory is loading...
Images of coconut directory is loading...
Images of macadamia directory is loading...
Images of brazil nuts directory is loading...
Images of cashews directory is loading...
Images of almonds directory is loading...
Images of pine nuts directory is loading...
Images of walnuts directory is loading...
```

```
[20]: total_Image_category_count=pd.DataFrame(Test_label_list).value_counts()
total_Image_category_count
```

```
[20]: almonds      5
brazil nuts      5
cashews          5
coconut          5
hazelnuts        5
macadamia        5
pecans           5
pine nuts        5
pistachios       5
walnuts          5
dtype: int64
```

```
[21]: lb = LabelBinarizer()
Test_y = lb.fit_transform(Test_label_list)
print(lb.classes_)
Test_y[1]
```

```
['almonds' 'brazil nuts' 'cashews' 'coconut' 'hazelnuts' 'macadamia'
 'pecans' 'pine nuts' 'pistachios' 'walnuts']
```

```
[21]: array([0, 0, 0, 0, 1, 0, 0, 0, 0, 0])
```

```
[ ]: print(f"total images in training data {len(training_image_list)} and its labels {len(training_label_list)}")
      print(f"total images in Validation data {len(Validation_image_list)} and its labels {len(Validation_label_list)}")
      print(f"total images in Testing data {len(Test_image_list)} and its labels {len(Test_label_list)}")
```

```
total images in training data 1163 and its labels 1163
total images in Validation data 50 and its labels 50
total images in Testing data 50 and its labels 50
```

```
[ ]: from sklearn.utils import shuffle
      training_image_list, train_y = shuffle(training_image_list, train_y, random_state=0)
      Test_image_list, Test_y = shuffle(Test_image_list, Test_y, random_state=0)
      Validation_image_list, validation_y = shuffle(Validation_image_list, validation_y, random_state=0)
```

```
[24]: !pip install ipyplot
```

```
Collecting ipyplot
  Downloading ipyplot-1.1.1-py3-none-any.whl (13 kB)
Requirement already satisfied: numpy in /opt/conda/lib/python3.7/site-packages (from ipyplot) (1.21.6)
Requirement already satisfied: IPython in /opt/conda/lib/python3.7/site-packages (from ipyplot) (7.33.0)
Requirement already satisfied: pillow in /opt/conda/lib/python3.7/site-packages (from ipyplot) (9.1.1)
Requirement already satisfied: shortuuid in /opt/conda/lib/python3.7/site-packages (from ipyplot) (1.0.9)
Requirement already satisfied: backcall in /opt/conda/lib/python3.7/site-packages (from IPython->ipyplot) (0.2.0)
Requirement already satisfied: matplotlib-inline in /opt/conda/lib/python3.7/site-packages (from IPython->ipyplot) (0.1.3)
Requirement already satisfied: pickleshare in /opt/conda/lib/python3.7/site-packages (from IPython->ipyplot) (0.7.5)
Requirement already satisfied: pexpect>4.3 in /opt/conda/lib/python3.7/site-packages (from IPython->ipyplot) (4.8.0)
Requirement already satisfied: traitlets>=4.2 in /opt/conda/lib/python3.7/site-packages (from IPython->ipyplot) (5.2.1.post0)
Requirement already satisfied: jedi>=0.16 in /opt/conda/lib/python3.7/site-packages (from IPython->ipyplot) (0.18.1)
Requirement already satisfied: prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0 in /opt/conda/lib/python3.7/site-packages (from IPython->ipyplot) (3.0.29)
Requirement already satisfied: setuptools>=18.5 in /opt/conda/lib/python3.7/site-packages (from IPython->ipyplot) (59.8.0)
Requirement already satisfied: decorator in /opt/conda/lib/python3.7/site-
```

```

packages (from IPython->ipyplot) (5.1.1)
Requirement already satisfied: pygments in /opt/conda/lib/python3.7/site-
packages (from IPython->ipyplot) (2.12.0)
Requirement already satisfied: parso<0.9.0,>=0.8.0 in
/opt/conda/lib/python3.7/site-packages (from jedi>=0.16->IPython->ipyplot)
(0.8.3)
Requirement already satisfied: ptyprocess>=0.5 in /opt/conda/lib/python3.7/site-
packages (from pexpect>4.3->IPython->ipyplot) (0.7.0)
Requirement already satisfied: wcwidth in /opt/conda/lib/python3.7/site-packages
(from prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0->IPython->ipyplot) (0.2.5)
Installing collected packages: ipyplot
Successfully installed ipyplot-1.1.1
WARNING: Running pip as the 'root' user can result in broken permissions
and conflicting behaviour with the system package manager. It is recommended to
use a virtual environment instead: https://pip.pypa.io/warnings/venv

```

```

[ ]: import ipyplot
ipyplot.plot_images(training_image_list,
    ↪max_images=20,img_width=200,labels=train_y)

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```

[26]: import ipyplot
ipyplot.plot_images(Validation_image_list,
    ↪max_images=10,img_width=200,labels=validation_y)

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```

[27]: import ipyplot
ipyplot.plot_images(Test_image_list, max_images=10,img_width=200,labels=Test_y)

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```

[ ]: training_image_list = np.array(training_image_list, dtype=np.float16) / 225.0
Test_image_list = np.array(Test_image_list, dtype=np.float16) / 225.0
Validation_image_list = np.array(Validation_image_list, dtype=np.float16) / 225.
    ↪0
training_image_list = training_image_list.reshape( -1,224,224,3)
Test_image_list = Test_image_list.reshape( -1,224,224,3)
Validation_image_list = Validation_image_list.reshape( -1,224,224,3)

```

```
[ ]: from keras.models import Sequential
from tensorflow.keras.layers import BatchNormalization
from keras.layers import Conv2D, MaxPooling2D, Activation, Flatten, Dropout,
↳Dense, LeakyReLU
```

```
[ ]: import tensorflow as tf
model = Sequential()
model.add(Conv2D(224, (3, 3), padding="same", input_shape=(224,224,3),
↳activation="relu"))
model.add(MaxPooling2D(pool_size=(3, 3)))
model.add(Conv2D(120, (3, 3), padding="same", activation="relu"))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(70, (3, 3), padding="same", activation="relu"))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(10, activation="softmax"))
model.summary()
```

2022-06-26 07:16:52.419358: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero

2022-06-26 07:16:52.540558: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero

2022-06-26 07:16:52.541291: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 224)	6272
max_pooling2d (MaxPooling2D)	(None, 74, 74, 224)	0
conv2d_1 (Conv2D)	(None, 74, 74, 120)	242040
max_pooling2d_1 (MaxPooling2D)	(None, 37, 37, 120)	0
conv2d_2 (Conv2D)	(None, 37, 37, 70)	75670
max_pooling2d_2 (MaxPooling2D)	(None, 18, 18, 70)	0
flatten (Flatten)	(None, 22680)	0

```

-----
dense (Dense)                (None, 25)                567025
-----
dense_1 (Dense)              (None, 10)                260
=====

```

```

Total params: 891,267
Trainable params: 891,267
Non-trainable params: 0
-----

```

```

2022-06-26 07:16:52.543057: I tensorflow/core/platform/cpu_feature_guard.cc:142]
This TensorFlow binary is optimized with oneAPI Deep Neural Network Library
(oneDNN) to use the following CPU instructions in performance-critical
operations:  AVX2 AVX512F FMA
To enable them in other operations, rebuild TensorFlow with the appropriate
compiler flags.
2022-06-26 07:16:52.543352: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero
2022-06-26 07:16:52.544024: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero
2022-06-26 07:16:52.544751: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero
2022-06-26 07:16:54.638634: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero
2022-06-26 07:16:54.639489: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero
2022-06-26 07:16:54.640130: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node
read from SysFS had negative value (-1), but there must be at least one NUMA
node, so returning NUMA node zero
2022-06-26 07:16:54.640725: I
tensorflow/core/common_runtime/gpu/gpu_device.cc:1510] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 15403 MB memory:  -> device:
0, name: Tesla P100-PCIE-16GB, pci bus id: 0000:00:04.0, compute capability: 6.0

```

```

[31]: import tensorflow as tf
print("--> Checking for physical Tensorflow devices")
for device in tf.config.list_physical_devices():

```

```
print("{} {}".format(device.name))
```

```
--> Checking for physical Tensorflow devices
```

```
: /physical_device:CPU:0
```

```
: /physical_device:GPU:0
```

```
[ ]: import tensorflow as tf
with tf.device('/GPU:0'):
    model.compile(optimizer='adam',loss = 'categorical_crossentropy',
    ↪metrics=['accuracy'])
    history = model.fit(training_image_list, train_y, batch_size = 32, epochs =
    ↪10,validation_data=(Validation_image_list,validation_y))
```

```
2022-06-26 07:16:56.866667: I
```

```
tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:185] None of the MLIR
Optimization Passes are enabled (registered 2)
```

```
Epoch 1/100
```

```
2022-06-26 07:16:58.357273: I tensorflow/stream_executor/cuda/cuda_dnn.cc:369]
```

```
Loaded cuDNN version 8005
```

```
37/37 [=====] - 12s 100ms/step - loss: 2.3128 -
```

```
accuracy: 0.1204 - val_loss: 2.3047 - val_accuracy: 0.1000
```

```
Epoch 2/100
```

```
37/37 [=====] - 3s 71ms/step - loss: 2.2946 - accuracy:
```

```
0.1384 - val_loss: 2.3015 - val_accuracy: 0.1000
```

```
Epoch 3/100
```

```
37/37 [=====] - 3s 72ms/step - loss: 2.2877 - accuracy:
```

```
0.1496 - val_loss: 2.3196 - val_accuracy: 0.1000
```

```
Epoch 4/100
```

```
37/37 [=====] - 3s 73ms/step - loss: 2.2669 - accuracy:
```

```
0.1384 - val_loss: 2.2793 - val_accuracy: 0.1000
```

```
Epoch 5/100
```

```
37/37 [=====] - 3s 74ms/step - loss: 2.2589 - accuracy:
```

```
0.1402 - val_loss: 2.3376 - val_accuracy: 0.1200
```

```
Epoch 6/100
```

```
37/37 [=====] - 3s 72ms/step - loss: 2.2057 - accuracy:
```

```
0.1909 - val_loss: 2.2584 - val_accuracy: 0.1400
```

```
Epoch 7/100
```

```
37/37 [=====] - 3s 71ms/step - loss: 2.1293 - accuracy:
```

```
0.2287 - val_loss: 2.2183 - val_accuracy: 0.1600
```

```
Epoch 8/100
```

```
37/37 [=====] - 3s 72ms/step - loss: 2.0205 - accuracy:
```

```
0.2752 - val_loss: 2.5473 - val_accuracy: 0.1400
```

```
Epoch 9/100
```

```
37/37 [=====] - 3s 71ms/step - loss: 1.8390 - accuracy:
```

```
0.3336 - val_loss: 2.0910 - val_accuracy: 0.3000
```

```
Epoch 10/100
```

37/37 [=====] - 3s 72ms/step - loss: 1.6347 - accuracy:
0.3783 - val_loss: 2.4089 - val_accuracy: 0.2600
Epoch 11/100
37/37 [=====] - 3s 71ms/step - loss: 1.3732 - accuracy:
0.4901 - val_loss: 2.3207 - val_accuracy: 0.2400
Epoch 12/100
37/37 [=====] - 3s 72ms/step - loss: 1.1430 - accuracy:
0.5761 - val_loss: 2.5717 - val_accuracy: 0.2200
Epoch 13/100
37/37 [=====] - 3s 72ms/step - loss: 0.9768 - accuracy:
0.6268 - val_loss: 3.0133 - val_accuracy: 0.2200
Epoch 14/100
37/37 [=====] - 3s 71ms/step - loss: 0.7715 - accuracy:
0.6965 - val_loss: 3.5198 - val_accuracy: 0.2000
Epoch 15/100
37/37 [=====] - 3s 72ms/step - loss: 0.7456 - accuracy:
0.7111 - val_loss: 3.4125 - val_accuracy: 0.2200
Epoch 16/100
37/37 [=====] - 3s 73ms/step - loss: 0.6158 - accuracy:
0.7610 - val_loss: 4.2818 - val_accuracy: 0.2200
Epoch 17/100
37/37 [=====] - 3s 75ms/step - loss: 0.5527 - accuracy:
0.7825 - val_loss: 4.5923 - val_accuracy: 0.2400
Epoch 18/100
37/37 [=====] - 3s 71ms/step - loss: 0.4536 - accuracy:
0.8186 - val_loss: 4.8601 - val_accuracy: 0.2400
Epoch 19/100
37/37 [=====] - 3s 71ms/step - loss: 0.3903 - accuracy:
0.8426 - val_loss: 5.8781 - val_accuracy: 0.2400
Epoch 20/100
37/37 [=====] - 3s 72ms/step - loss: 0.0983 - accuracy:
0.9570 - val_loss: 15.3657 - val_accuracy: 0.2200
Epoch 68/100
37/37 [=====] - 3s 71ms/step - loss: 0.0991 - accuracy:
0.9561 - val_loss: 15.7765 - val_accuracy: 0.1800
Epoch 69/100
37/37 [=====] - 3s 71ms/step - loss: 0.0958 - accuracy:
0.9579 - val_loss: 16.0560 - val_accuracy: 0.2000
Epoch 70/100
37/37 [=====] - 3s 72ms/step - loss: 0.1000 - accuracy:
0.9587 - val_loss: 15.8149 - val_accuracy: 0.1800
Epoch 71/100
37/37 [=====] - 3s 71ms/step - loss: 0.0985 - accuracy:
0.9596 - val_loss: 15.6338 - val_accuracy: 0.1800
Epoch 72/100
37/37 [=====] - 3s 72ms/step - loss: 0.0947 - accuracy:
0.9604 - val_loss: 15.8038 - val_accuracy: 0.2000
Epoch 73/100

37/37 [=====] - 3s 71ms/step - loss: 0.0932 - accuracy: 0.9587 - val_loss: 15.9161 - val_accuracy: 0.2000
Epoch 74/100
37/37 [=====] - 3s 72ms/step - loss: 0.0945 - accuracy: 0.9613 - val_loss: 16.1149 - val_accuracy: 0.2000
Epoch 75/100
37/37 [=====] - 3s 71ms/step - loss: 0.0956 - accuracy: 0.9587 - val_loss: 15.8416 - val_accuracy: 0.2000
Epoch 76/100
37/37 [=====] - 3s 71ms/step - loss: 0.1010 - accuracy: 0.9570 - val_loss: 15.5053 - val_accuracy: 0.2200
Epoch 77/100
37/37 [=====] - 3s 71ms/step - loss: 0.0996 - accuracy: 0.9596 - val_loss: 15.9069 - val_accuracy: 0.2200
Epoch 78/100
37/37 [=====] - 3s 72ms/step - loss: 0.0935 - accuracy: 0.9613 - val_loss: 14.7761 - val_accuracy: 0.1800
Epoch 79/100
37/37 [=====] - 3s 74ms/step - loss: 0.1024 - accuracy: 0.9596 - val_loss: 13.5144 - val_accuracy: 0.2000
Epoch 80/100
37/37 [=====] - 3s 71ms/step - loss: 0.1473 - accuracy: 0.9458 - val_loss: 13.2087 - val_accuracy: 0.2400
Epoch 81/100
37/37 [=====] - 3s 71ms/step - loss: 0.1610 - accuracy: 0.9433 - val_loss: 14.9309 - val_accuracy: 0.1800
Epoch 82/100
37/37 [=====] - 3s 71ms/step - loss: 0.2634 - accuracy: 0.9183 - val_loss: 10.2234 - val_accuracy: 0.2000
Epoch 83/100
37/37 [=====] - 3s 73ms/step - loss: 0.2031 - accuracy: 0.9355 - val_loss: 12.4378 - val_accuracy: 0.1800
Epoch 84/100
37/37 [=====] - 3s 71ms/step - loss: 0.1624 - accuracy: 0.9450 - val_loss: 10.9212 - val_accuracy: 0.1800
Epoch 85/100
37/37 [=====] - 3s 71ms/step - loss: 0.1707 - accuracy: 0.9424 - val_loss: 13.8902 - val_accuracy: 0.1800
Epoch 86/100
37/37 [=====] - 3s 71ms/step - loss: 0.2568 - accuracy: 0.9166 - val_loss: 10.6315 - val_accuracy: 0.2000
Epoch 87/100
37/37 [=====] - 3s 72ms/step - loss: 0.1872 - accuracy: 0.9424 - val_loss: 12.6840 - val_accuracy: 0.1800
Epoch 88/100
37/37 [=====] - 3s 71ms/step - loss: 0.1433 - accuracy: 0.9501 - val_loss: 12.0299 - val_accuracy: 0.1600
Epoch 89/100

```

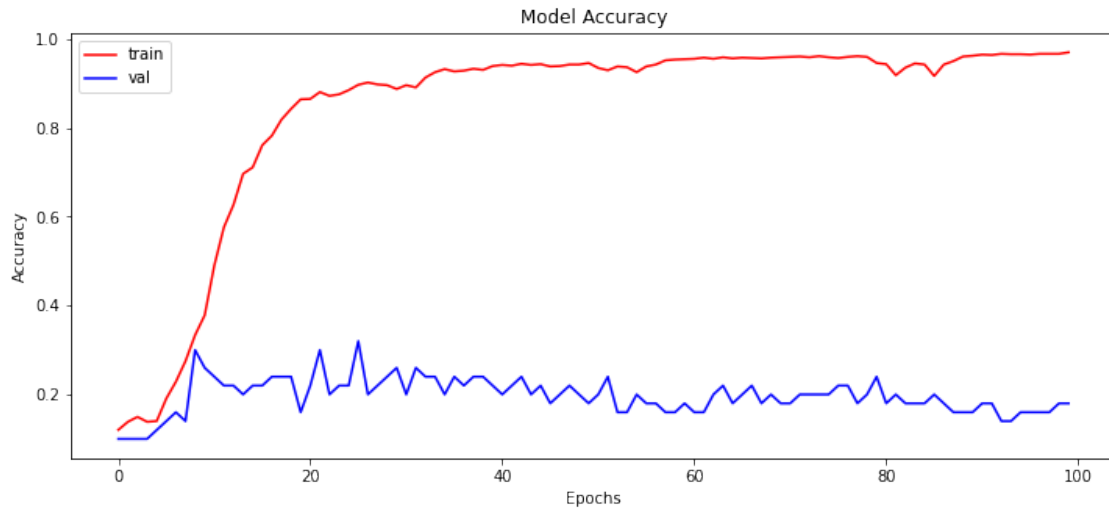
37/37 [=====] - 3s 71ms/step - loss: 0.0955 - accuracy:
0.9604 - val_loss: 14.8394 - val_accuracy: 0.1600
Epoch 90/100
37/37 [=====] - 3s 71ms/step - loss: 0.0977 - accuracy:
0.9622 - val_loss: 13.1870 - val_accuracy: 0.1600
Epoch 91/100
37/37 [=====] - 3s 73ms/step - loss: 0.0876 - accuracy:
0.9647 - val_loss: 15.2232 - val_accuracy: 0.1800
Epoch 92/100
37/37 [=====] - 3s 72ms/step - loss: 0.0874 - accuracy:
0.9639 - val_loss: 14.7531 - val_accuracy: 0.1800
Epoch 93/100
37/37 [=====] - 3s 71ms/step - loss: 0.0792 - accuracy:
0.9665 - val_loss: 15.5327 - val_accuracy: 0.1400
Epoch 94/100
37/37 [=====] - 3s 71ms/step - loss: 0.0795 - accuracy:
0.9656 - val_loss: 15.7015 - val_accuracy: 0.1400
Epoch 95/100
37/37 [=====] - 3s 72ms/step - loss: 0.0797 - accuracy:
0.9656 - val_loss: 15.5856 - val_accuracy: 0.1600
Epoch 96/100
37/37 [=====] - 3s 71ms/step - loss: 0.0774 - accuracy:
0.9647 - val_loss: 16.1991 - val_accuracy: 0.1600
Epoch 97/100
26/37 [=====>...] - ETA: 0s - loss: 0.0734 - accuracy:
0.9688

```

```

[36]: plt.figure(figsize=(12, 5))
plt.plot(history.history['accuracy'], color='r')
plt.plot(history.history['val_accuracy'], color='b')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epochs')
plt.legend(['train', 'val'])
plt.show()

```



```
[ ]: plt.figure(figsize=(12, 5))
plt.plot(history.history['loss'], color='r')
plt.plot(history.history['val_loss'], color='b')
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epochs')
plt.legend(['train', 'val'])
plt.show()
```

